

LAPORAN PRAKTIKKUM
TEKNOLOGI IoT

PERTEMUAN 3 Jaringan Nirkabel dan Komunikasi HTTP



Nama: Novita Syifa Azhara
NPM: 25782078
Kelas: 3C

Dosen: Dr. Ir. Septafiansyah Dwi Putra, S.T., M.T.
Agus Ambarwari, S.Pd., M.Kom.

TEKNOLOGI REKAYASA INTERNET
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI LAMPUNG

3.5.1 Praktikum 1: NodeMCU sebagai Web Server (Menyajikan HTML)

Skenario: Anda merakit NodeMCU sebagai server pintar. Modul ini membaca sensor DHT dan menampilkan suhunya di sebuah halaman web (UI), sekaligus menyediakan tombol di web tersebut untuk menghidupkan/mematikan lampu LED.

Langkah Kerja Rangkaian:

Hubungkan pin DATA / OUT / S DHT ke pin D4 (GPIO 2). (Pastikan VCC DHT ke 3V3, GND ke GND).

Hubungkan kaki Anoda (kaki panjang) LED ke pin D6 (GPIO 12). Hubungkan kaki Katoda (kaki pendek) LED ke salah satu kaki Resistor 220 Ohm, lalu hubungkan kaki resistor lainnya ke pin GND NodeMCU. (Catatan: LED ini dikendalikan oleh pin D6 / GPIO 12 yang pada kode program didefinisikan sebagai relayPin).

Langkah Kerja Pemrograman:

Buat berkas baru (New Sketch). Tuliskan kode program di bawah. Jangan lupa mengganti variabel ssid dan password!

Catatan teknis: Kita menggunakan teknik Raw String Literal C++ (R"rawliteral(...)rawliteral") untuk menulis kode antarmuka HTML multibaris, sehingga kita tidak repot mengetikkan tanda kutip "" di setiap baris HTML.

Blink | Arduino IDE 2.3.10

File Edit Sketch Tools Help

NodeMCU 1.0 (ESP-12E...)

Blinkino

```
1 #include <ESP8266WiFi.h>
2 #include <ESP8266WebServer.h>
3 #include <DHT.h>
4
5 const char* ssid = "samsung galaxy";
6 const char* password = "ailanadianti0";
7
8 ESP8266WebServer server(80);
9
10 const byte dhtPin = 2;
11 const byte relayPin = 12;
12 DHT dht(dhtPin, DHT22);
13
14 const char index_html[] PROGMEM = R"rawliteral(
15 <!DOCTYPE html>
16 <html>
17 <head>
18 <meta name="viewport" content="width=device-width, initial-scale=1">
19 <title>IoT Dashboard</title>
20 <style>
21   body { font-family: Arial; text-align: center; margin-top: 50px; }
22   button { padding: 15px 30px; font-size: 20px; border-radius: 8px; margin: 10px; cursor: pointer; }
23   .btn-on { background-color: #4CAF50; color: white; border: none; }
24   .btn-off { background-color: #f44336; color: white; border: none; }
25   .sensor-box { font-size: 24px; font-weight: bold; }
26 </style>
27 </head>
28 <body>
29 <div ESP8266 Web Server</div>
30 <div class="sensor-box">
31   <p>Suhu Saat Ini: <strong>XTEMPERATUREX</strong> Celcius</p>
32 </div>
33 <div><strong>Relay</strong></div>
34 <a href="/relay/on">button class="btn-on">ON</button></a>
35 <a href="/relay/off">button class="btn-off">OFF</button></a>
36 </body>
37 </html>
38 )rawliteral";
39
40 void handleRoot() {
41   String html = index_html; // Salin kerangka HTML ke variabel dinamis
42   float t = dht.readTemperature();
43
44   // Mengganti teks placeholder XTEMPERATUREX dengan suhu nyata
45   if (!isnan(t)) {
46     html.replace("XTEMPERATUREX", "-.-");
47   } else {
48     html.replace("XTEMPERATUREX", String(t));
49   }
50
51   server.send(200, "text/html", html);
52 }
53
54 void handleRelayOn() {
55   digitalWrite(relayPin, HIGH);
56   server.sendHeader("Location", "/");
57   server.send(303);
58 }
59
60 void handleRelayOff() {
61   digitalWrite(relayPin, LOW);
62   server.sendHeader("Location", "/");
63   server.send(303);
64 }
65
66 void setup() {
67   Serial.begin(115200);
68   pinMode(relayPin, OUTPUT);
69   digitalWrite(relayPin, LOW);
70   dht.begin();
71
72   WiFi.mode(WIFI_STA);
73   WiFi.begin(ssid, password);
74   while (WiFi.status() != WL_CONNECTED) { delay(500); Serial.print("."); }
75   Serial.println("\nIP Address Server Anda: ");
76   Serial.println(WiFi.localIP());
77
78   server.on("/", handleRoot);
79   server.on("/relay/on", handleRelayOn);
80   server.on("/relay/off", handleRelayOff);
81   server.begin();
82 }
83
84 void loop() {
85   server.handleClient();
86 }
```

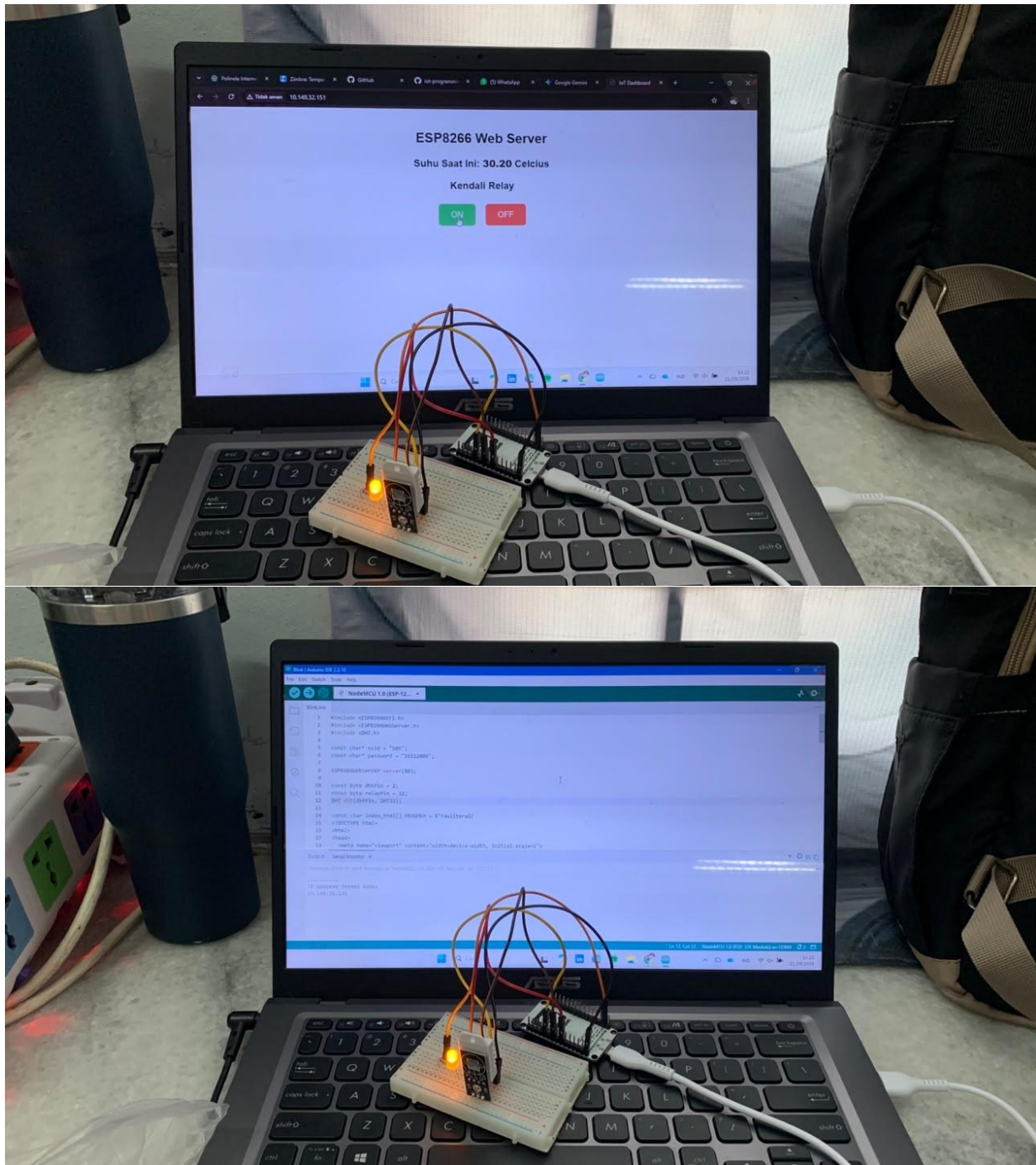
Serial Monitor X

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3')

New Line 115200 baud

...
IP Address Server Anda:
10.244.199.31

Ln 24, Col 74 - NodeMCU 1.0 (ESP-12E Module) on COM3



Penjelasan Singkat Kode:

ESP8266WebServer server(80);: Menginisialisasi library Server agar terus mendengarkan panggilan di jalur lalu lintas standar web (Port 80).

html.replace("%TEMPERATURE%", String(t));: Fitur manipulasi teks (String). Perintah ini secara cerdas akan menyapu dokumen HTML, mencari teks placeholder %TEMPERATURE%,

dan menggantinya dengan variabel angka suhu yang didapat dari sensor aktual sebelum dikirim ke layar pengunjung. Inilah esensi dari halaman web dinamis.

`server.on("/relay/on", handleRelayOn);`: Fungsi Routing. Jika ada klien mengunjungi tautan URL berakhiran `/relay/on`, server akan menjalankan aksi mengubah state LED (menyalakan/mematikan output), lalu me-redirect layar pengunjung kembali ke halaman depan (Location: `/`) via kode 303.

`server.handleClient();`: Berada di dalam `loop()` untuk terus-menerus mengecek apakah ada permintaan pengunjung yang datang setiap milidetiknya.

3.5.2 Praktikum 2: NodeMCU sebagai HTTP Client (Meminta Layanan)

Skenario: Anda sekarang memprogram NodeMCU kedua (atau meminta teman kelompok di sebelah Anda) untuk menjadi Client. Perangkat Klien ini dipasang LDR. Jika LDR tertutup kegelapan, ia akan mengirim sinyal HTTP secara jarak jauh (via jaringan Wi-Fi) untuk menyalakan LED di Server milik teman Anda (Praktikum 1).

Langkah Kerja Rangkaian:

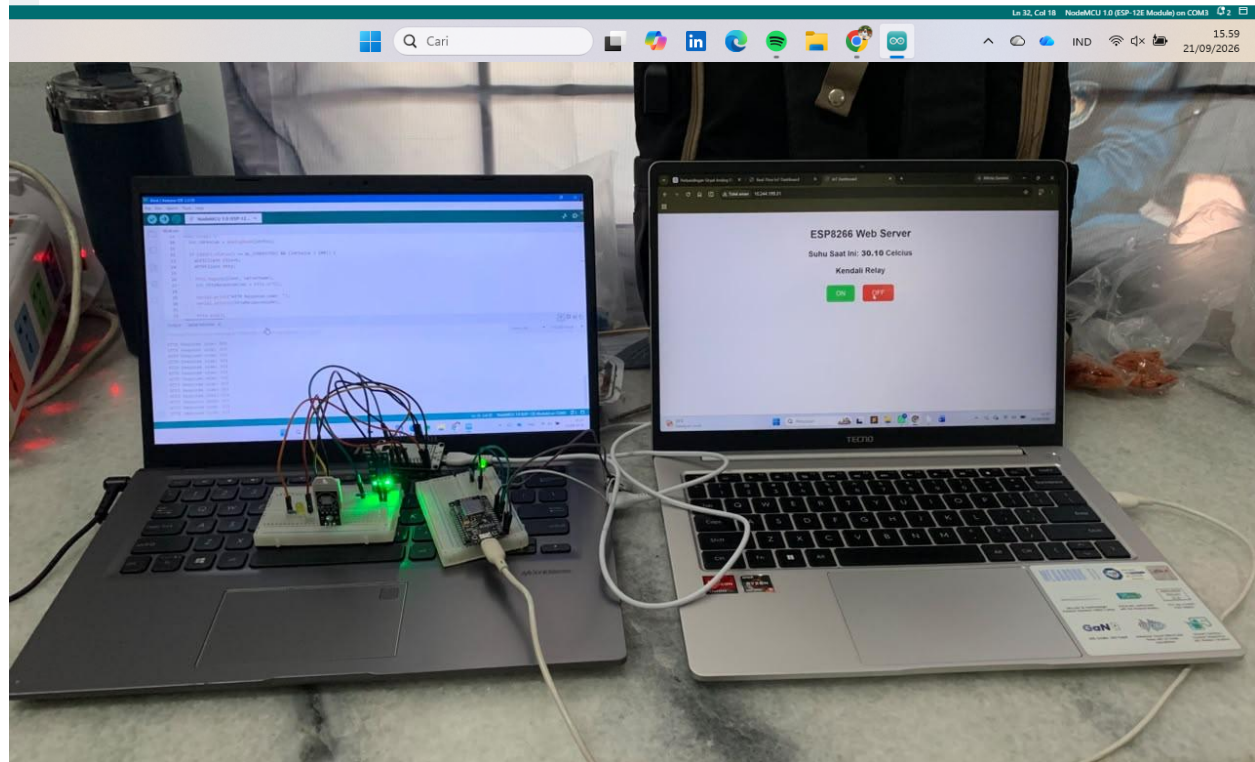
Rangkai sensor LDR dan Resistor 10k Ohm sebagai pembagi tegangan (seperti di Modul 2) lalu hubungkan jalur sinyalnya ke pin A0.

```
Blink | Arduino IDE 2.3.10
File Edit Sketch Tools Help
NodeMCU 1.0 (ESP-12...)
Blinkino
1 #include <ESP8266WiFi.h>
2 #include <ESP8266HTTPClient.h>
3 #include <WiFiClient.h>
4
5 const char* ssid = "samsung galaxy";
6 const char* password = "allanadiant18";
7
8 const char* serverName = "http://10.244.199.31/relay/on";
9 const byte idrPin = A0;
10
11 void setup() {
12   Serial.begin(115200);
13   WiFi.mode(WIFI_STA);
14   WiFi.begin(ssid, password);
15   while (WiFi.status() != WL_CONNECTED) { delay(500); Serial.print("."); }
16   Serial.println("\nClient Terhubung ke Wi-Fi!");
17 }
18
19 void loop() {
20   int idrValue = analogRead(idrPin);
21
22   if ((WiFi.status() == WL_CONNECTED) && (idrValue > 100)) {
23     WiFiClient client;
24     HTTPClient http;
25
26     http.begin(client, serverName);
27     int httpResponseCode = http.GET();
28
29     Serial.print("HTTP Response code: ");
30     Serial.println(httpResponseCode);
31
32     http.end();
33     delay(10000);
34     delay(2000);
35   }
36 }
```

Output: Serial Monitor x

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3')

HTTP Response code: 200
HTTP Response code: 200
HTTP Response code: 200



3.6 Latihan

Kerjakan soal berikut secara mandiri untuk menguji penalaran dan modifikasi jaringan.

Latihan 1 (Pemahaman Jaringan): Berdasarkan wawasan dari subbab 3.3.1, sebutkan secara logis mengapa kita menggunakan mode jaringan Station (STA) pada kedua praktikum di atas, dan bukan mode Access Point (AP)? Kita menggunakan mode Station (STA) agar kedua perangkat (NodeMCU Server dan NodeMCU Client atau HP/Laptop) dapat terhubung ke satu jaringan router/hotspot yang sama (satu *subnet*). Dalam arsitektur *Client-Server* lokal, kedua perangkat wajib berada dalam jaringan yang sama agar bisa saling mengenali alamat IP lokal satu sama lain. Jika menggunakan mode *Access Point (AP)*, NodeMCU akan bertindak sebagai pemancar mandiri. Akibatnya, perangkat lain harus terus-menerus memutuskan koneksi dari internet utama/hotspot untuk bergabung ke jaringan AP tersebut, dan komunikasi antar NodeMCU (seperti Client mengakses Server milik teman) akan menjadi jauh lebih sulit karena terisolasi pada jaringan yang berbeda.

Latihan 2 (Modifikasi Antarmuka & Telemetry): Pada Praktikum 1, tampilan web saat ini baru menyajikan data suhu ruangan dan masih menggunakan dua tombol terpisah (ON dan OFF). Lakukan dua modifikasi berikut pada Web Server:

Menampilkan Kelembapan: Tambahkan pembacaan kelembapan menggunakan fungsi `dht.readHumidity()`. Sisipkan placeholder `%HUMIDITY%` pada kerangka HTML di dalam tag `<div class="sensor-box">` (contoh: `<p>Kelembapan: <strong>%HUMIDITY%</strong>%</p>`), lalu perbarui fungsi `handleRoot()` menggunakan `html.replace("%HUMIDITY%", ...)` agar nilai kelembapan muncul di halaman web.

Tombol Dinamis Tunggal (Toggle Button): Ubah antarmuka agar hanya menggunakan 1 buah tombol tunggal yang statusnya adaptif mengikuti kondisi LED saat ini:

Jika LED sedang OFF, tombol menampilkan tulisan "NYALAKAN (ON)" (misal: warna hijau/biru) dan mengarahkan aksi ke `/relay/on`.

Jika LED sedang ON, tombol otomatis berubah menjadi tulisan "MATIKAN (OFF)" (warna merah) dan mengarahkan aksi ke `/relay/off`.

(Petunjuk: Anda dapat menyematkan placeholder baru di HTML seperti `%RELAY_BUTTON%`, lalu di fungsi `handleRoot()`, periksa status pin relay/LED menggunakan `digitalRead(relayPin)` sebelum mengganti placeholder tersebut dengan elemen tautan tombol yang sesuai).

Tuliskan potongan baris kode HTML dan fungsi `handleRoot()` yang Anda ubah!

Blinkino

```
1 #include <ESP8266WiFi.h>
2 #include <ESP8266WebServer.h>
3 #include <DHT.h>
4
5 const char *ssid = "samsung galaxy";
6 const char *password = "ailanadianti0";
7
8 ESP8266WebServer server(80);
9 const byte dhtPin = 2; // Pin data DHT11 (GPIO 2 / D4)
10 const byte relayPin = 12; // Pin kontrol LED/Relay (GPIO 12 / D6)
11 DHT dht(dhtPin, DHT22);
12
13 const char index_html[] PROGMEM = R"rawliteral(
14 <!DOCTYPE html>
15 <html>
16 <head>
17 <meta name="viewport" content="width=device-width, initial-scale=1">
18 <title>IoT Dashboard - Latihan 2</title>
19 <style>
20   body { font-family: Arial; text-align: center; margin-top: 50px; }
21   .btn { padding: 15px 30px; font-size: 20px; border-radius: 8px; margin: 10px; cursor: pointer; text-decoration: none; display: inline-block; color: white; border: none; }
22   .btn-on { background-color: #4CAF50; }
23   .btn-off { background-color: #F44336; }
24   .sensor-box { font-size: 24px; font-weight: bold; margin: 15px 0; }
25 </style>
26 </head>
27 <body>
28   <h1>SERVER NOVITA</h1>
29   <div class="sensor-box">
30     <p>Suhu Saat Ini: <strong>%TEMPERATURE%</strong> &deg;C</p>
31     <p>Kelembapan: <strong>%HUMIDITY%</strong> %</p>
32   </div>
33   <h2>Kendali Relay</h2>
34   %RELAY_BUTTON%
35 </body>
36 </html>
37 )rawliteral";
38
39 void handleRoot() {
40   String html = index_html;
41
42   // Membaca suhu dan kelembapan dari sensor DHT
43   float t = dht.readTemperature();
44   float h = dht.readHumidity();
45
46   // Penggantian placeholder suhu
47   if (isnan(t)) {
48     html.replace("%TEMPERATURE%", "--");
49   } else {
50     html.replace("%TEMPERATURE%", String(t));
51   }
52
53   // 1. Penggantian placeholder kelembapan
54   if (isnan(h)) {
55     html.replace("%HUMIDITY%", "--");
56   } else {
57     html.replace("%HUMIDITY%", String(h));
58   }
59
60   // 2. Penggantian placeholder tombol dinamis tunggal (Toggle Button)
61   String relayButtonHtml = "";
62   if (digitalRead(relayPin) == HIGH) {
63     // Jika LED sedang menyala, tampilkan tombol MATIKAN (OFF)
64     relayButtonHtml = "<a href=\"/relay/off\"><button class=\"btn btn-off\">MATIKAN (OFF)</button></a>";
65   } else {
66     // Jika LED sedang mati, tampilkan tombol NYALAKAN (ON)
67     relayButtonHtml = "<a href=\"/relay/on\"><button class=\"btn btn-on\">NYALAKAN (ON)</button></a>";
68   }
69 }
```

```
70     html.replace("%RELAY_BUTTON%", relayButtonHtml);
71
72     server.send(200, "text/html", html);
73 }
74
75 void handleRelayOn() {
76     digitalWrite(relayPin, HIGH);
77     server.sendHeader("Location", "/");
78     server.send(303);
79 }
80
81 void handleRelayOff() {
82     digitalWrite(relayPin, LOW);
83     server.sendHeader("Location", "/");
84     server.send(303);
85 }
86
87 void setup() {
88     Serial.begin(115200);
89     pinMode(relayPin, OUTPUT);
90     digitalWrite(relayPin, LOW);
91     dht.begin();
92
93     WiFi.mode(WIFI_STA);
94     WiFi.begin(ssid, password);
95     while (WiFi.status() != WL_CONNECTED) {
96         delay(500);
97         Serial.print(".");
98     }
99     Serial.println("\nIP Address Server Anda: ");
100    Serial.println(WiFi.localIP());
101
102    server.on("/", handleRoot);
103    server.on("/relay/on", handleRelayOn);
104    server.on("/relay/off", handleRelayOff);
105    server.begin();
106 }
107
108 void loop() {
109     server.handleClient();
110 }
```

Output Serial Monitor x

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3')

New Line 115200 baud

```
.....
IP Address Server Anda:
10.244.199.151
...
IP Address Server Anda:
10.244.199.151
```

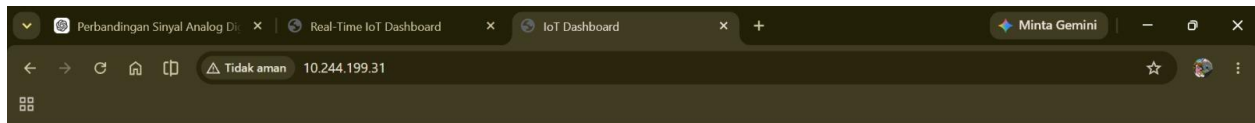
Latihan 3 (Analisis Kode Client): Pada Praktikum 2, Serial Monitor Client mencetak HTTP Response code. Analisislah apa yang akan dicetak oleh Serial Monitor Anda jika ternyata NodeMCU Server teman Anda mendadak kehilangan daya (offline)? (Silakan cabut daya Server dan lihat hasil erornya). Jika server mati (*offline*) atau tidak dapat dijangkau, fungsi `http.GET()` akan gagal melakukan koneksi ke URL tujuan. Serial Monitor akan mencetak **angka negatif** (biasanya -1 atau kode error bawaan dari library `HTTPClient` yang menandakan kegagalan koneksi/timeout), alih-alih kode status sukses seperti 200 atau 303.

Latihan 4 (Pengembangan Algoritma): Anda ingin mendesain Klien Pendeteksi Suhu. Modifikasi conditional block `if` di Praktikum 2 sehingga Client mengirim permintaan (GET Request) HANYA JIKA sensor DHT membaca suhu melebihi 35 Celcius (bukan menggunakan LDR). Tulis baris algoritmanya.

```

1  #include <ESP8266WiFi.h>
2  #include <ESP8266HTTPClient.h>
3  #include <WiFiClient.h>
4  #include <DHT.h>
5
6  const char *ssid = "samsung galaxy";
7  const char *password = "ailanadiant18";
8  const char *serverName = "http://http://10.244.199.151//relay/on"; // Ganti dengan IP Server teman Anda
9
10 const byte dhtPin = 2; // Pin data DHT11 (GPIO 2 / D4)
11 DHT dht(dhtPin, DHT22);
12
13 void setup() {
14     Serial.begin(115200);
15     dht.begin();
16
17     WiFi.mode(WIFI_STA);
18     WiFi.begin(ssid, password);
19
20     while (WiFi.status() != WL_CONNECTED) {
21         delay(500);
22         Serial.print(".");
23     }
24     Serial.println("\nClient Terhubung ke Wi-Fi!");
25 }
26
27 void loop() {
28     // Membaca suhu dari sensor DHT (menggantikan IdrValue dari Praktikum 2)
29     float t = dht.readTemperature();
30
31     // Cek apakah pembacaan sensor valid
32     if (isnan(t)) {
33         Serial.println("Gagal membaca sensor DHT!");
34         delay(2000);
35         return;
36     }
37
38     Serial.print("Suhu saat ini: ");
39     Serial.print(t);
40     Serial.println(" C");
41
42     // Perintah Latihan 4: Kirim HTTP GET HANYA JIKA Wi-Fi terhubung DAN suhu > 35
43     if ((WiFi.status() == WL_CONNECTED) && (t > 35.0)) {
44         WiFiClient client;
45         HTTPClient http;
46
47         http.begin(client, serverName);
48         int httpResponseCode = http.GET();
49
50         Serial.print("Suhu > 35C! HTTP Response code: ");
51         Serial.println(httpResponseCode);
52
53         http.end();
54
55         // Cooldown agar tidak spam request ke server
56         delay(10000);
57     }
58
59     delay(2000);
60 }

```

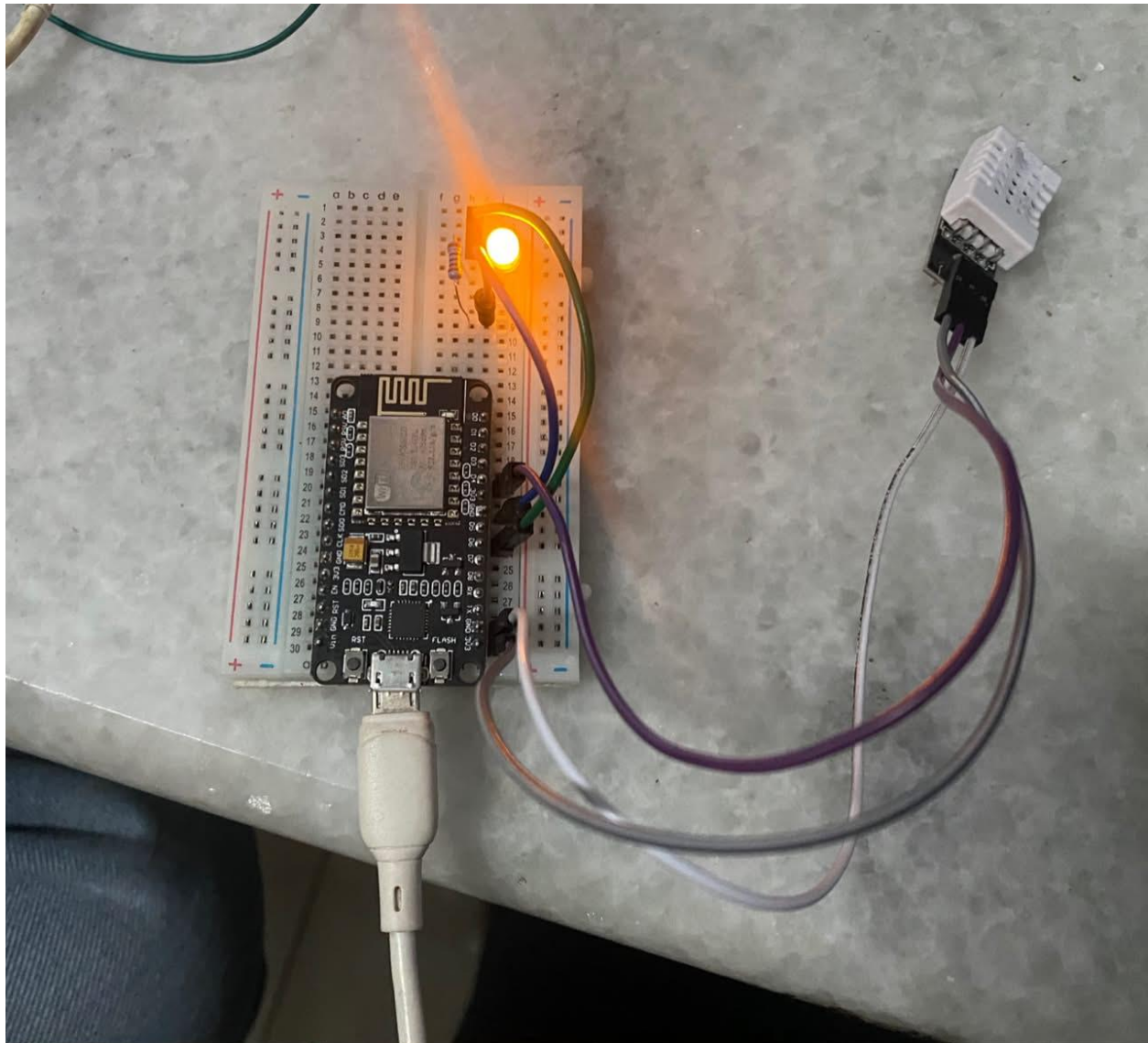


ESP8266 Web Server

Suhu Saat Ini: **1.30** Celcius

Kendali Relay





3.7 Tugas: Pengembangan Web Server Cerdas dan Analisis Arsitektur

Antarmuka (User Interface) IoT tidak boleh statis. Saat ini, Web Server kita tidak mampu memperbarui informasi suhunya secara otomatis ke layar pengunjung tanpa diperintah. Kita harus menyelesaikannya.

Skenario Proyek (Web Dashboard): Anda akan bertindak sebagai Full-Stack IoT Developer untuk memutakhirkan kapabilitas Web Server pada Latihan 2, sembari mengevaluasi limitasi arsitektur tradisional HTTP.

Tujuan: Mampu memanipulasi struktur kerangka HTML mentah yang tersimpan di memori NodeMCU untuk mencapai interaktivitas tampilan otomatis (Auto-Refresh).

Tingkat Kesulitan: Menengah

Instruksi:

Lakukan modifikasi kode hanya pada sisi NodeMCU Server (Latihan 2).

Modifikasi variabel Raw Literal `index_html`. Sisipkan perintah Meta Refresh HTML (sebuah Tag standar web kuno namun ampuh) persis di dalam cakupan tag `<head>` untuk memaksa browser memuat ulang halaman secara sekuensial setiap 5 detik.

Hint (Petunjuk): Sintaks standar yang harus disisipkan adalah `<meta http-equiv="refresh" content="5">`.

Setelah selesai, buka browser, dan amati layarnya. Biarkan selama 20 detik tanpa menyentuh mouse/keyboard.

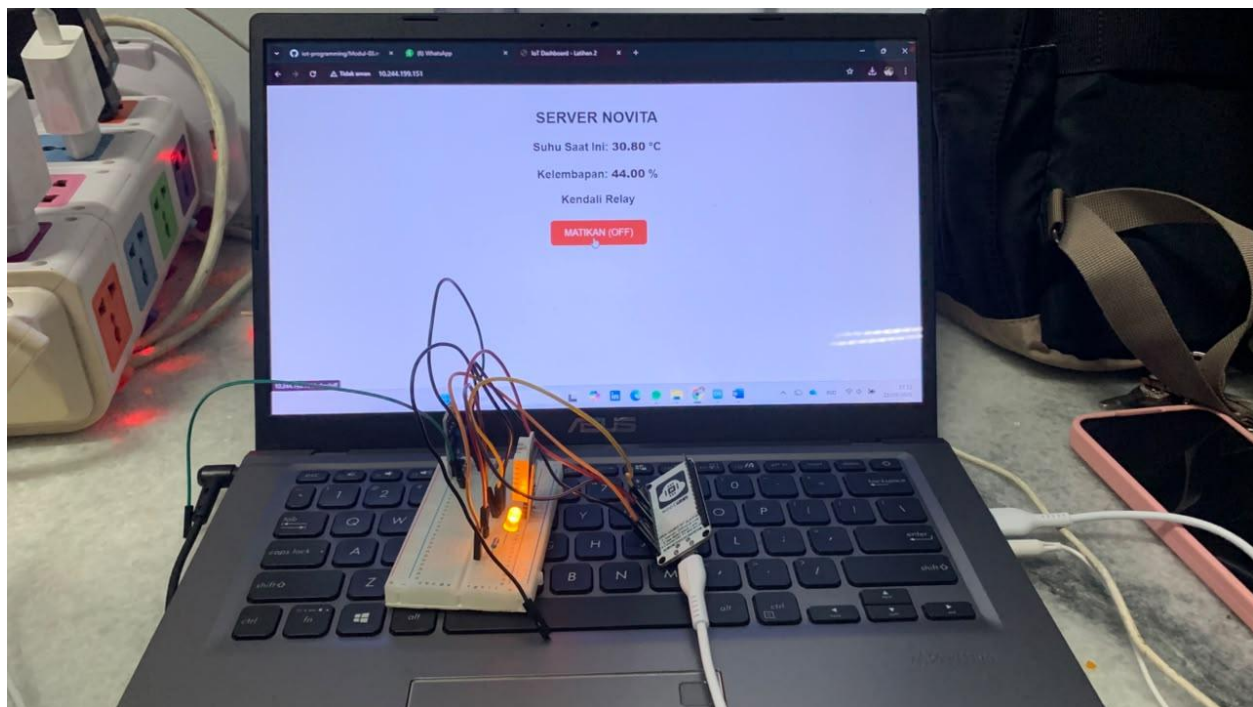
Instruksi Analisis Arsitektur:

Setelah Auto-Refresh berjalan, cobalah tekan tombol ON / OFF (kendali LED) dari browser. Apa kelemahan visual/interaksi (flickering) yang Anda alami saat menggunakan metode Meta Refresh HTTP konvensional seperti ini untuk sistem yang mengklaim diri sebagai Real-Time Dashboard? Tulis evaluasi logis Anda di Laporan Praktikum (Anda bisa membahas masalah bandwidth, layar berkedip putih, atau potensi bentrokan jika Anda mengeklik tepat di detik ke-5).

Kriteria Keberhasilan:

Nilai suhu dan kelembapan di browser otomatis berganti seiring kondisi lingkungan tanpa perlu mengeklik tombol refresh di browser.

Jawaban analisis mengenai kelemahan protokol HTTP GET statis tertuang jelas di dokumentasi tugas.



1. Tujuan Praktikum

- Memahami cara memanipulasi struktur kerangka HTML mentah (*Raw String Literal*) yang tersimpan di memori NodeMCU ESP8266.
- Mengimplementasikan fitur pembaruan tampilan otomatis (*Auto-Refresh*) pada antarmuka *Web Server IoT*.

- Mampu menganalisis limitasi dan kelemahan arsitektur tradisional HTTP menggunakan metode *Meta Refresh* konvensional.

2. Implementasi dan Modifikasi Kode

Untuk mencapai interaktivitas tampilan otomatis tanpa perlu mengeklik tombol *refresh* secara manual di peramban (*browser*), dilakukan penyisipan tag standar HTML *Meta Refresh* di dalam bagian `<head>` pada variabel *Raw Literal* `index_html` di sisi NodeMCU Server.

3. Hasil Pengamatan dan Uji Coba

Berdasarkan pengujian perangkat keras dan tampilan antarmuka pada *dashboard* web:

- Pembaruan Data Otomatis: Nilai sensor suhu (tercatat 30.80 °C) dan kelembapan (44.00 %) berhasil berganti secara berkala setiap 5 detik tanpa intervensi manual pada tombol *refresh* peramban.
- Kendali Relay (Toggle Button): Tombol interaktif berhasil menyesuaikan status secara dinamis, menampilkan opsi "MATIKAN (OFF)" ketika perangkat menyala.
- Kondisi Perangkat: Rangkaian NodeMCU ESP8266 terhubung dengan sensor dan modul penunjang di atas *breadboard* berfungsi dengan baik dalam melayani *request* HTTP klien.

4. Analisis Arsitektur & Evaluasi *Meta Refresh*

Penggunaan metode *Meta Refresh* konvensional (`<meta http-equiv="refresh" content="5">`) pada sistem *Real-Time Dashboard* memiliki beberapa kelemahan mendasar dari sisi teknis:

- Gejala Layar Berkedip (*Flickering*): Setiap kali durasi 5 detik tercapai, *browser* akan memuat ulang seluruh halaman secara penuh (*full page reload*). Hal ini menyebabkan layar mengalami kedipan putih sejenak yang mengganggu kenyamanan visual pengguna.
- Boros Penggunaan *Bandwidth*: Karena halaman dimuat ulang sepenuhnya, server harus mengirim ulang seluruh elemen statis (seperti kode CSS, struktur HTML, teks, hingga judul) secara berulang-ulang, meskipun data yang benar-benar berubah hanya nilai suhu dan kelembapan saja.
- Potensi Bentrokan Waktu (*Race Condition / Conflict*): Jika pengguna mengeklik tombol kendali (seperti tombol *ON/OFF*) tepat pada detik ke-5 bersamaan dengan momen *browser* memicu otomatisasi *meta refresh*, perintah klik pengguna bisa saja terganggu, tertunda, atau mengalami kegagalan eksekusi sesaat karena bentrok dengan proses muat ulang halaman.

5. Kesimpulan

Penambahan tag *Meta Refresh* berhasil membuat *Web Server* NodeMCU menyajikan data secara otomatis (*auto-refresh*). Kendati demikian, metode ini kurang optimal jika diaplikasikan pada sistem IoT berskala besar atau murni *real-time* karena memicu masalah *flickering* dan inefisiensi *bandwidth*. Sebagai solusi tingkat lanjut di masa depan, sistem dapat ditingkatkan menggunakan komunikasi asinkron berbasis AJAX atau protokol ringan seperti WebSocket untuk memperbarui data tanpa perlu memuat ulang seluruh halaman web.